

WEEK - 2

pom.xml

<dependencies>

<dependency>

<groupId>junit</groupId>

<artifactId>junit</artifactId>

<version>3.8.1</version>

<scope>test</scope>

</dependency>

<dependency>

<groupId>javax.servlet</groupId>

<artifactId>javax.servlet-api</artifactId>

<version>4.0.1</version>

<scope>provided</scope>

</dependency>

<dependency>

<groupId>org.seleniumhq.selenium</groupId>

<artifactId>selenium-java</artifactId>

<version>4.27.0</version>

</dependency>

<dependency>

<groupId>org.testing</groupId>

<artifactId>testing</artifactId>

<version>7.10.2</version>

<scope>test</scope>

</dependency>

</dependencies>

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-surefire-plugin</artifactId>

<version>3.5.2</version>

</plugin>

maven junit dependency
3.8.1

java servlet api
maven dependency

selenium java
maven dependency
4.27.0

maven testing
dependency
7.10.2

maven
surefire
plugin
3.5.2

< plugin >

< groupId > org.apache.maven.plugins < /groupId >

< artifactId > maven-compiler-plugin < /artifactId >

< version > 3.13.0 < /version >

< configuration >

< source > 17 < /source >

< target > 21 < /target >

< /configuration >

< /plugin >

< /plugins >

< finalName > 6229 < /finalName >

< /build >

< /project >

maven
compiler
plugin

Our
own

- ① download Eclipse zip file and extract the contents of the eclipse file.
- ② Create a maven project from Eclipse as :-
 - click on File in left corner → New → Maven Project
 - Click Next • Search for org.apache.maven.archetypes and select webapp file
 - In groupId & artifactId, type appropriately & select war as package
 - Click Finish • Type Y & press enter.
- ③ Open the pom.xml file and add your dependencies.
- ④ Right click on project name → Build Path → Configure build path → Targeted Runtimes → If Apache Tomcat 9 not there, download the zip file & unzip it → New → Apache → Apache Tomcat V9. Select the folder, apply, apply & close.
- ⑤ After adding dependencies → Right click on pom.xml → source → Format Project → clean → Your project
- ⑥ Go to Users ~~name~~, in student, go to Eclipse workspace → MyEclipse1 → src → main → Create new folder java. In that, in src, create a new folder 'test' & in it another folder. Right click on project → Refresh java
- ⑦ In Eclipse → Help → Eclipse Marketplace → drag & drop the testing file from Chrome (gotten when searching for 'testing for eclipse') → Confirm → Accept → Trust selected / install by searching for it.

⑧ Right click on project → Team → Share project → Tick on Use or create Repository → select the project → Create Repository → Finish

⑨ Again, right click on project name → Team → Commit → Move all files from unstaging to staging by clicking on "+".
Write a commit message. Set author & committer as name < email >.
Commit → Push Head → Copy & paste the link of new repository created in your GH account → Username & Password

⑩ In GH, click on Profile → Settings → developer settings → Tokens (classic) → generate New Token → GNT (classic()) → Note as MyDevToken1 → No expiration → select repo → generate token → copy token.

⑪ In eclipse, in user your username & in password, paste the generated token. Push & repeat the same again. You will see the files in GH.

⑫ src/test/java → Right click → New → class, enter name of test class, tick ~~psvm~~, type package com.cmr.it.tests; → In error list click on move to - - -

⑬ Right click on com.cmr.it.tests package → Team → Commit → Move from unstaged to staged → write commit message → Commit & Push → configure → save & Push.

WEEK - 3

- ① Create an AWS Free tier account.
- ② Then, download Mobaxterm → Choose Home Edition → Installer Type
- ③ Sign in to your AWS account → Sign in using Root user email → Enter your mail and password → download Google Authenticator and enter the code if MFA code is asked.
- ④ Select region as Mumbai. Click on EC2 Instance. Go to Instances in the left side panel.
To stop & kill an instance, select that instance by tick marking it, in instance state → stop instance. After it stops in instance state → Terminate instance
- ⑤ To create a new instance, select New/Launch Instance. Enter a name for it (Jenkins) → Select Ubuntu → Select Ubuntu 22.04 as machine image in instance type select t2 micro.
- ⑥ Next, under Key pair, press create a new key pair → give name (Jenkins-demo-B11) • give Key pair type as RSA and private key file format as .pem
- ⑦ Then, in Network settings, click on Edit. Here, in Inbound security group Rules, click on Add security group rule.
Type - Custom TCP ; Port Range - 8080 ; source type - Anywhere / custom as source, select 0.0.0.0/0 for source type
- ⑧ Click on Launch Instance and in instances, refresh it to view the instance.
- ⑨ In Mobaxterm → click on session → click on SSH. Tick mark the wanted instance & copy that IP address, paste it in Remote Host in Mobaxterm - Tick mark specify username and type 'ubuntu'.
- ⑩ Next, click on Advanced SSH setting → Tick mark use private key. Select the Browse option in its box and in downloads; select the pem file - Click on OK - Click on Accept
- ⑪ Type the command of clear.

~\$ sudo su - (to go to root directory)

⑫ Type jenkins.io in a browser, go to the first site and click on download. Download Jenkins 2.412.1 LTS for → Ubuntu → Long Term Support release commands

~# sudo apt-get update

~# sudo wget -O

----- .key (the first 2 lines)

~# echo

----- (the 3, 4 & 5th lines)

----- >/dev/null

⑬ To install java, scroll down & under installation of java

~# sudo apt update

~# sudo apt install fontconfig openjdk -y

~# java --version (21x, 8 & 21 ✓)

⑭ To install Jenkins server

~# sudo apt-get update

~# sudo apt-get install jenkins -y

~# jenkins --version

⑮ To connect the server, scroll down and under Start Jenkins

~# sudo systemctl enable jenkins

~# sudo systemctl start jenkins

~# sudo systemctl status jenkins (Screen shot 1) *

⑯ Copy the Instance IP address; paste in a browser with ":8080"

The first line after Group is the password. If forgotten then stop the server using Ctrl+z and type sudo cat "paste the command in the web page"

⑰ Install the predefined/recommended plugins. In Create first admin user

→ Give admin for username, password. Give full name & email address → Save & Continue → Save & Finish.

⑱ Start using Jenkins (Screen shot 2) *

WEEK - 5 (ANSIBLE)

- ① Open AWS → In EC2 → Launch Instance. Follow the same steps as for week 3 till Network Settings. Keep name as 'ansible-demo-ssh'.
In Network settings, Edit → Add security group rule
 - i) Type - Custom TCP, Port Range - 8080, Source type - Custom Source - 0.0.0.0/0
 - ii) Type - All ICMP - IPv4, Source type - AnywhereChange Number of Instance - 3. Click on launch instance.
↳ Ansible server, server 1, server 2
- ② Instances → Reload it to see the instances. Rename the 2nd instance to 'server1'. Rename the 3rd instance to 'server2'.
- ③ In Metaxterm, repeat the same steps as in week 2 for 'ansible-demo-ssh' instance.
 - Switch to root > `$ sudo su -`
 - Update packages > `$ apt update -y`
 - To include official project's PPA > `$ apt-add-repository ppa:ansible/ansible`
 - Update again > `$ apt update`
 - Install ansible > `$ apt install ansible -y`
- ④ Check python version > `$ python3 --version`
Check Ansible version > `$ ansible --version`
- ⑤ Enter command `$ nano /etc/hosts`
In it,
 - Paste 'server1 public IP' & type server1
 - Paste 'server2 public IP' & type server2~~Ctrl~~ Ctrl + O to save → Ctrl + X to exit
- ⑥ Enter the command
 - ~# cd .ssh/
 - ~# ls
 - ~# ssh-keygen -t rsaPress Enter → Enter → Enter.

7 Enter command `# cat id_rsa.pub`
Copy the key that is in that file.

8 Connect the server1 in AWS. In CLI, type `$ nano .ssh/authorized_keys`
Here, paste the copied key from MobaXterm here. `Ctrl+O` → Enter → `Ctrl+X`.

9 Connect to the server2 in AWS. In CLI, type `$ nano .ssh/authorized_keys`
Paste the key copied from MobaXterm here. `Ctrl+O` → Enter → `Ctrl+X`.

10 Enter the command

Then, `# cd ..`
`# ping server1` → `Ctrl+Z`
Next, `# ping server2` → `Ctrl+Z`

11 Next, enter command.

`# ssh ubuntu@65.0.32.176 (server1 public IP)` → yes
`ubuntu@ip-172-31-7-206: $ logout`

Then, `# ssh ubuntu@13.233.156.176 (server2 public IP)` → yes
`$ logout` (Both should show last login)

12 After.

`# mkdir ansible`
`# cd ansible`
`# nano inventory`
Here, copy & paste matter in document that is
[webservers]
server1
server2
`Ctrl+O` → Enter → `Ctrl+X`
`# cat inventory`
`# nano ansible.cfg`
Here, copy & paste matter from document
[defaults]

`ask_pass=false`
`Ctrl+O` → Enter → `Ctrl+X`

13 Then, enter command `# nano install_webservers.yml`

Here, copy & paste matter from document

`- name: install web server`

`when: inventory_hostname == 'server2'`

14 Then, use the same series of steps starting from `$ nano etc/hosts` but change the IP address to private ones.

15 Enter "`ansible-playbook -i` ..." command. Open server1 public IP address to see nginx & server2 public IP in browser to see Apache (SS1*) (SS2*)

WEEK - 54 (WAR file) Metaxterm

- ① Create EC2 instance. Repeat same steps.
In Network settings, edit →
 - i) Type → HTTP, source type → custom, source → 0.0.0.0/0
 - ii) Type → custom, source type → custom, source → 0.0.0.0/0, port → 8080
 - iii) Type → AllKMP, IPv4, source type → Anywhere
- ② Follow same process as Exp-3
- ③ Click on New Item. Item name → HelloWorldJob. Select Freestyle project.
OK.
- ④ Type Hello World Job in description. Select Build steps → Execute shell. In that type: `echo "Hello World CMKIT"`. Click Apply & then save.
Uptime
- ⑤ Click on Build Now.
- ⑥ Configure Git in Jenkins:
Install git on Jenkins instance
Install github plugin on Jenkins GUI
Configure ^{git} ~~install~~ on Jenkins GUI
- ⑦ In Metaxterm, type "`sudo apt-get install git`". Then "`git --version`".
In Jenkins GUI → Dashboard → Manage Jenkins → Plugins → Available Plugins
→ Github Integration → select it → Install → back to top page
Again → Manage Jenkins → Tools → look for git → Apply & save.
- ⑧ Dashboard → New Item. Keep item name as PullCodeFromGithub. Select Freestyle project then OK.
- ⑨ Type "Pull Code From Github". Select Git in Source Code Management
- ⑩ Login to Github account. Go to devops Pipeline. Select code & copy the HTTPS repo link. Paste it in Repository URL. Apply & save.
Click Build Now.
- ⑪ In Metaxterm, type
 - `cd /var/lib/jenkins/workspace`
 - `ll`
 - `cd PullCodeFromGithub`
 - `ll`
- ⑫ To go to root directory automatically, type "`cd ~`"
- ⑬ Integrate Maven with Jenkins:
 - Setup Maven on Jenkins server
 - Setup environment variables

=> JAVA_HOME, M2, M2_HOME

- Install Maven plugin
- Configure Maven & Java

⑭ Go to google and type maven download. Go in to the first link that appears. Copy the link address of Binary tar.gz archive (link).

⑮ Type the below commands

- cd /opt
- ll
- wget "linkaddress"
- ll
- tar -xvzf apache-maven-3.9.9-bin.tar.gz
- mv apache-maven-3.9.9 maven
- cd maven
- cd bin
- ll

⑯ Go back to root using "cd ~". Type "ll -a". Edit the environmental variables by going to the profile file using command "vi .profile".

⑰ .profile

→ Enter 'i' to write commands

M2_HOME = /opt/maven

M2 = /opt/maven/bin

JAVA_HOME = /usr/lib/jvm/java-17-openjdk-amd64

Uses specific environment and startup programs

PATH = \$PATH:\$HOME/bin:\$JAVA_HOME:\$M2_HOME:\$M2

export PATH

→ Press esc & then type ':wq' to exit the file

⑱ Type command "source .profile" & then "echo \$PATH" to get the paths of all environmental variables

⑲ Manage Jenkins → Plugins → Available Plugins → Maven Integration. Select it and install.

⑳ Manage Jenkins → Tools →

- i) Add JDK : Name as 'jdk-17' & source as /usr/lib.(JAVA_HOME path)
- ii) Add Maven : Name as 'maven 3.9.9' & source as /opt/maven

20) dashboard → New Item → Enter Item name as "FirstMavenProject".
Select item type as "Maven Project". Click OK.

21) Keep Git as source code Management. Paste the git repository link.
In Build, Goals & Options, enter "clean install".
Apply & save.

WEEK - 4 (WAR file) GitBash

- ① Create EC2 instances the same as previous exp with name jenkins-devm. Change the no. of instances to 2 and launch it
- ② Change the name of 2nd one to jenkins-sitm.
- ③ Select jenkins-devm instance & click on Connect. Go to SSH client and copy the "Example: command" below
- ④ Go to Downloads. Right-click and click on Open Git Bash. ~~Copy~~ Paste the above copied command, enter "yes".
- ⑤ Type the command

```
sudo su -
```

Go to Jenkins download in Chrome → Download → Stable Release: Ubuntu/Debian → Long Term Support release

Copy and paste the first three commands: `sudo`, `echo`, `sudo apt-get update`

For Java installation type "`sudo apt install fontconfig openjdk-17-jre`".

"`sudo apt update`"

Then, install jenkins, type "`sudo apt-get install jenkins`"
- ⑥ Then, execute all three commands of start Jenkins: `enable`, `start`, `status`.
- ⑦ Copy the public IP address of AWS instance, paste it in new tab in browser: 8080 → enter. Paste the password from GitBash. Next, install suggested plugins
- ⑧ Manage Jenkins → Plugins → Available plugins → GitHub integration: select & install
- ⑨ Manage Jenkins → Tools. Under git installations, change Name to Git → Apply & save.
- ⑩ Type commands in GitBash: "`cd /opt`" "`ll`" (no maven)
Go to maven download → download Apache maven → In files copy the link address of Binary tar.gz archive (link)
"`wget {paste the link address}`"
"`ll`" (copy the maven file name (in red))
"`tar x paste the file name`" (to unzip it)
"`mv apache-maven-3.9.9 maven`" (copy from ll)
"`ll`" (see changed name)

"cd maven"

"cd bin"

"./mvn --version"

"cd ~" (to come back directly to root directory)

"cd .." (come out from this symbol) (x)

"ll -a"

"vi .profile"

⑪ Click i to go to insert mode

M2-HOME = /opt/maven

M2 = /opt/maven/bin

JAVA-HOME = /usr/lib/jvm/java-17-openjdk-amd64

PATH = \$PATH : \$HOME/bin : \$JAVA-HOME : \$M2-HOME : \$M2

export PATH

Press esc and type :wq to exit the file.

⑫ Type mvn --version to see that its version can also be accessed here

⑬ Manage Jenkins → Plugins → Available Plugins → Maven Integration :
Select & Install → Restart Jenkins (check that option)

⑭ Manage Jenkins → Tools

i) In JDK installations → Add JDK

Name : JAVA-17

JAVA-HOME : /usr/lib/jvm/java-17-openjdk-amd64

ii) In Maven installations → Add Maven → Uncheck install automatically

Name : Maven-3.9.9

MAVEN-HOME : /opt/maven

Apply → Save

⑮ New Item → Name : FirstMavenProject → Type : Maven project → OK

Description : First Maven Project

Source Code Management : Git

Go to your Github → Go to devPipeline → Code : Copy the HTTPS URL
Paste in Repository URL

In Build : Goals and options → "clean install"

To get java path

- Right click gitbash title → Open New Window

- Paste the instance SSH key & press enter

- sudo su -

- find / -name jvm

- cd /usr/lib/jvm

- find / -name java-17* (copy 1st line)

WEEK - 6

- ① Continue with previous experiment. Start both the Jenkins instances and deploy the Jenkins server.
- ② Copy the SSH key of Jenkins - sitm and paste in Jit Bash.
- ③ Then, type the below commands:
" sudo apt-get update "
" sudo apt-get install -y tomcat9 "
" sudo apt-get install tomcat9-admin "
- ④ Take the Public IP address of the Jenkins - sitm and add 8080.
You will get the web page containing "It works!"
- ⑤ Go to the below path "cd /etc/tomcat9 "
" ll "
- ⑥ To add users ~~to~~, go to tomcat-users.xml file with the command:
" sudo vim tomcat-users.xml "
- ⑦ Go to insert mode by pressing "i"
Type the below
<user username="dishu" password="dishu1234" roles="manager-script,
manager-status, manager-gui" />
Press "Esc" and type ":wq" to exit the file
- ⑧ Restart the tomcat service using "sudo service tomcat9 restart"
- ⑨ Go to Jenkins Dashboard, Manage Jenkins → plugins → available plugins
→ Select "deploy to containers" → Install.
- ⑩ In dashboard, click on drop down option next to First Match Project
and click on Configure.
- ⑪ Under Post-Build actions, select 'deploy war/ear to a container'
WAR/EAR files - **/*.war
Context path - sit
Add container - Tomcat 9x Remote

Add Credentials:

Username - disha (that you put in file before)

Password - disha1234

Tomcat URL - "IP of sitm instance:8080" (http://152.66.246.38:8080/)

② Apply & save

③ go to the "IP of sitm instance:8080/sit". You should see Hello world.

config

2017/10/10

→ 1. Add the trigger
→ 2. Add the trigger
→ 3. Add the trigger
→ 4. Add the trigger
→ 5. Add the trigger

configure → trigger → add trigger

WEEK - 7

① Create a new folder "Exp-7" on desktop. In it, right click, → show more options → Open gitBash.

② These, type command

"git clone [paste the web URL of the git repo development]"

③ Follow the below commands :

```
$ ll
$ cd MyDevPipeline1
$ ll
$ cd src
$ ll
$ cd main
$ ll
$ cd webapp
$ ll
$ vim index.jsp
```

Make changes to the index.jsp. Change "Hello World" to "Welcome to CMR Institute of Technology"

④ Next, push changes to github using below commands

\$git push origin master

```
$ git status
```

```
$ git add.
```

```
$ git status
```

```
$ git commit -m "Modified index.jsp"
```

```
$ git config user.email "dixha05301@gmail.com"
```

```
$ git config user.name "dixha0532"
```

In the pop up that appears, select token. In github, go to Settings → Developer Settings → Personal Access Tokens → Tokens (classic)

→ Generate new token → Generate new token (classic) → Note: Exp7

→ In scopes, select repo → Click on Generate token.

Copy & paste this generated token in pop up.

⑤ In Jenkins dashboard, near project name, click on drop down menu, click on Configure.

⑥ Select Trigger → Check Poll SCM → Enter "* * * * *" Apply & save.

⑦ The build will start automatically and the output in the ":8080/ait" will also be updated.